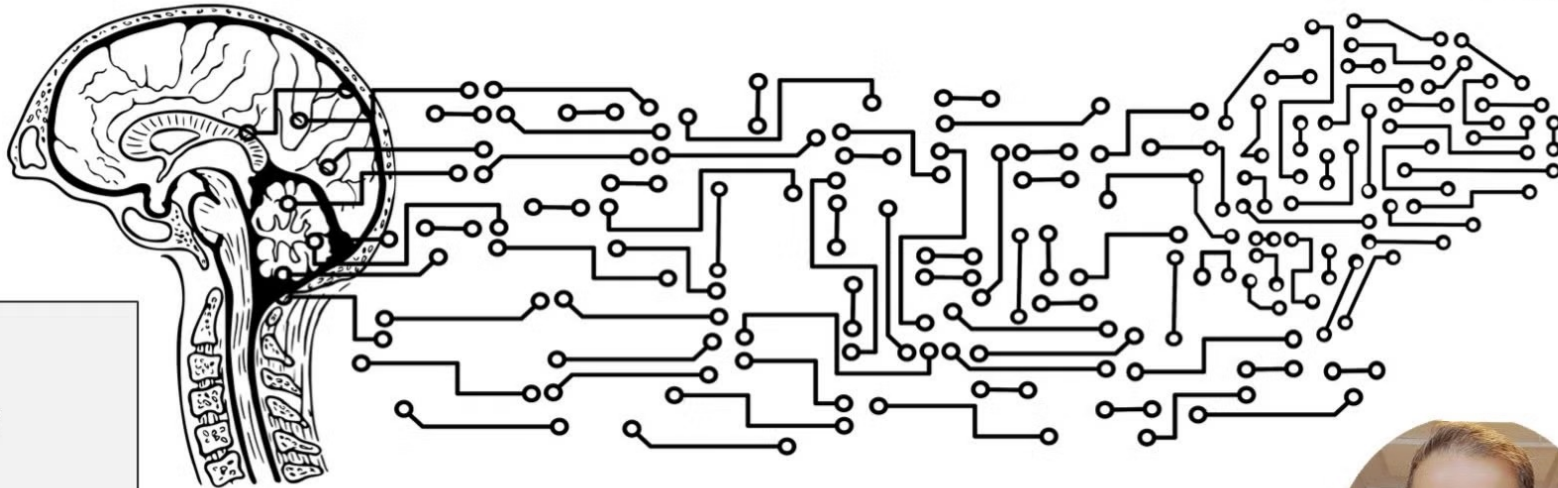


CYBERSECURITY AND AI USERGROUP



Meetup 010

Wed 06 May 26
03:30 - 04:30
Online

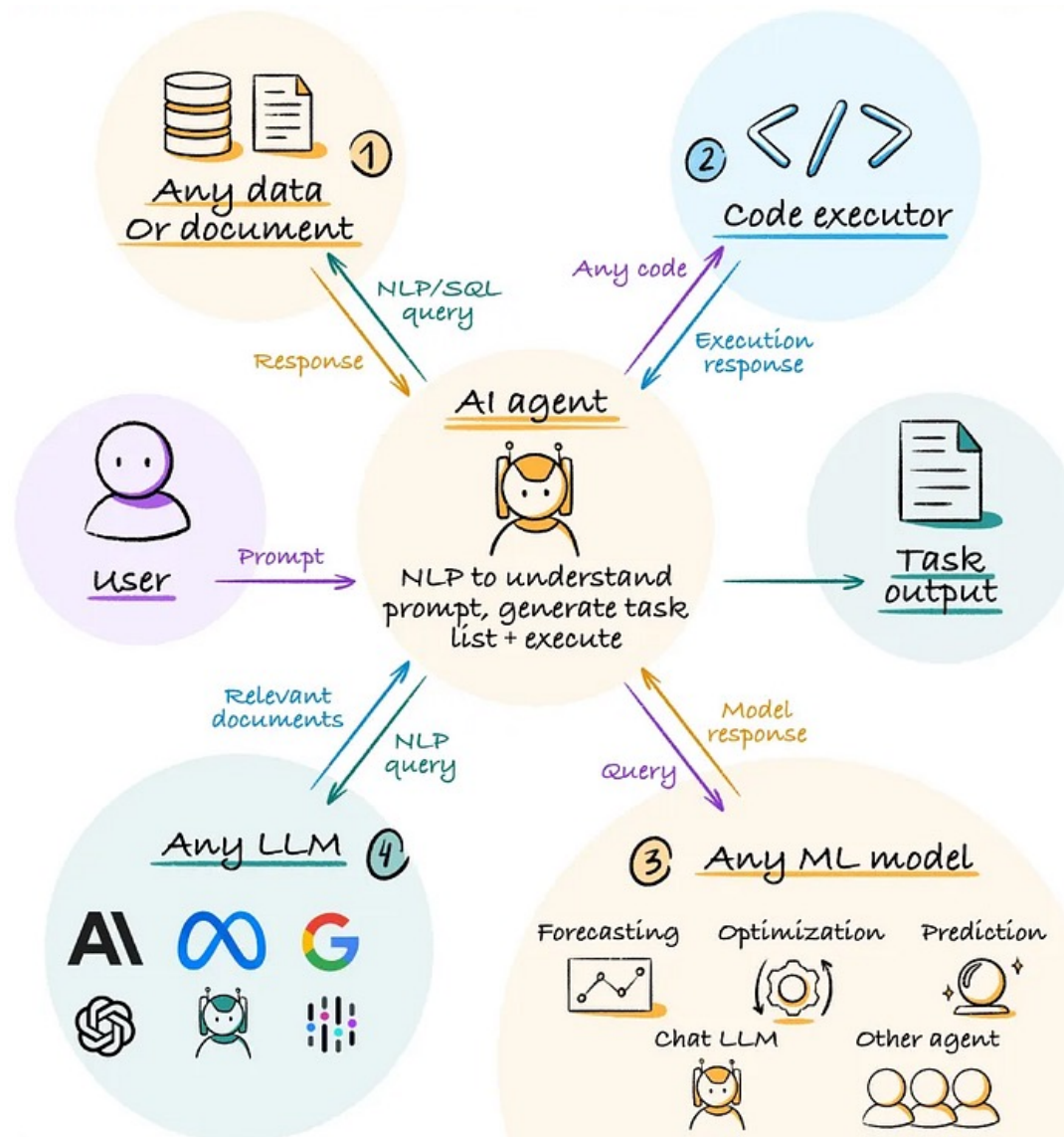
Join: <https://rb.gy/a8czxk>

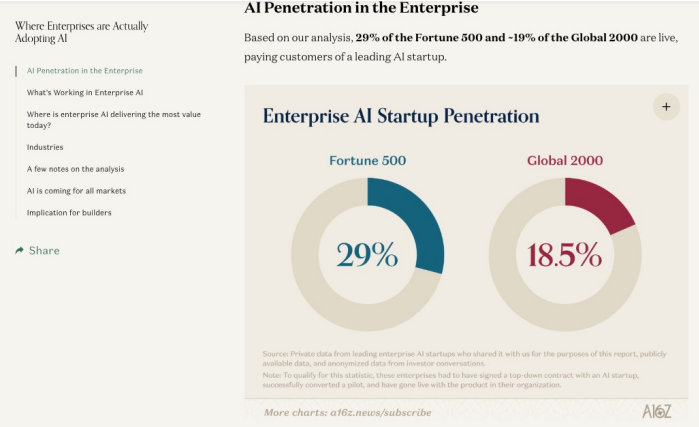
User Group Details:
[https://www.meetup.com/
cybersecurity-and-ai-
usergroup/](https://www.meetup.com/cybersecurity-and-ai-usergroup/)

Agent Thinking, Reasoning & System Architecture



Mohit Sharma
Chief Consultant
Atea AS





Aaron Levie on AI's Enterprise Adoption

17K views · 9 months ago

a16z

a16z General Partner Martin Casado sits down with Box cofounder and CEO Aaron Levie to talk about how AI is changing not just ..



19 chapters Introduction to AI in the Enterprise | Aaron Levy, CEO of Box | AI in the Enterprise: Challenges and...

Generative AI

Why AI Adoption Stalls, According to Industry Data

by Erin Eatough, Keith Ferrazzi, Wendy Smith and Shonna Waters

February 17, 2026

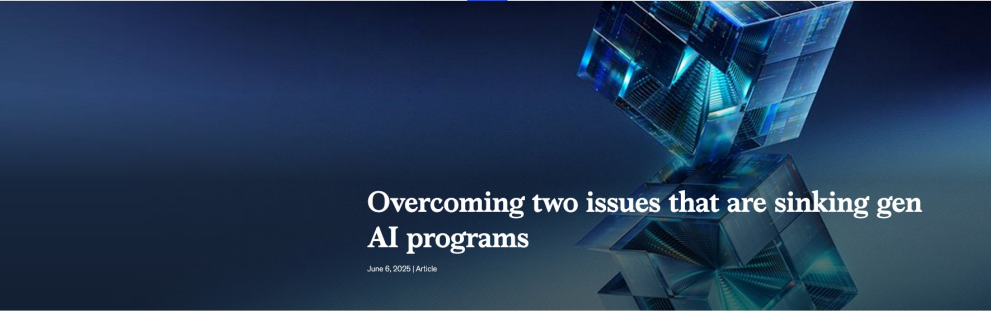


Juan Moyano/Getty Images

Companies in most industries are investing heavily in artificial intelligence: [88% of companies reporting regular AI use](#). Yet many leaders report familiar frustrations. AI adoption stalls. Performance gains plateau. Employees experiment with new tools but don't integrate them deeply into how work actually gets done, leaving executives increasingly concerned about ROI.

McKinsey & Company

Tech & AI
How We Help Clients
[Our Insights](#)
Case Studies
Contact Us



Overcoming two issues that are sinking gen AI programs

June 6, 2025 | Article

By [Curt Jacobsen](#), Erik Witte, Kaz Kazmier, and Oscar Villarreal

Share Print Download Save

Hard experience has revealed common technology pitfalls in building a gen AI capability and proven strategies for overcoming them.

Evidence	Archetype	Constraints	Boundaries	Patterns	Variability	Architecture	Economics	Failures	Security	Governance
----------	-----------	-------------	------------	----------	-------------	--------------	-----------	----------	----------	------------

THE CURRENT REALITY: AI THEATER

CURIOSITY, EXPERIMENTS,
AND FRAGMENTED VALUE

EXPERIMENTS
EVERYWHERE



62% Experimenting,
23% Scaled
MCKINSEY FINDINGS

PILOTS
DON'T SCALE



Adoption Stalls
HBR INSIGHTS

NO MEASURABLE
ENTERPRISE VALUE

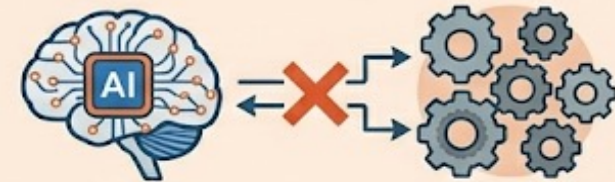


Unclear Outcome
& Value
CITRIX PERSPECTIVE

**SUMMARY: THEY DON'T KNOW
HOW TO USE IT PROPERLY.**

THE HIDDEN PROBLEM: OLD WAYS OF WORKING

THE BARRIER IS
TREAT AI AS A TOOL, NOT A SYSTEM



IS IT A TOOL OR A SYSTEM?
AS A TOOL (a Wrench) | (Interconnected Nodes)

SO, IT BREAKS
BOLTED-ON PROCESSES FAIL



THE INTERN METAPHOR
HIRING INTERNS, THEN FORCING THEM
TO FOLLOW OUTDATED MANUALS



AI Agents vs Chatbots vs Multi-Agent

AI Agents

Systems that use models to **think** and **act** toward specific goals **autonomously**



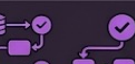
Manily Nila
manily.nila@aol.com
AI Agent Certification

Chatbots

Systems designed for **conversation**, handling **basic** interactions and queries

Multi-agents

Multiple AI agents working **together, coordinating** to solve complex problems or achieve **shared goals**.

	 AI CHATBOT (Conversational)	 AI AGENT (Autonomous)	 AI WORKFLOW (Structured)
AUTONOMY	 Low: Reacts per prompt	 High: Self-directs	 Medium: Follows fixed path
DECISION-MAKING	 Per Message	 Dynamic, adapts, multi-step	 Bounded by pre-design
MEMORY	 Within a session	 Long-term context, vector stores	 State passed between steps
TOOL USE	 None / Minimal	 Vast Toolset	 Defined Pipeline Tools pipeline nodes
HUMAN INVOLVEMENT	 Drives every turn	 Goal-setter, agent acts	 Designs the flow
RELIABILITY	Predictable within turn	Variable - Can loop	High - Deterministic path
BEST FOR & EXAMPLES	 Customer support bot	 Claude Code	 n8n

CHOOSING AUTONOMY LEVELS: BOUNDED VS OPEN-ENDED EXECUTION

BOUNDED EXECUTION (CONSTRAINED)

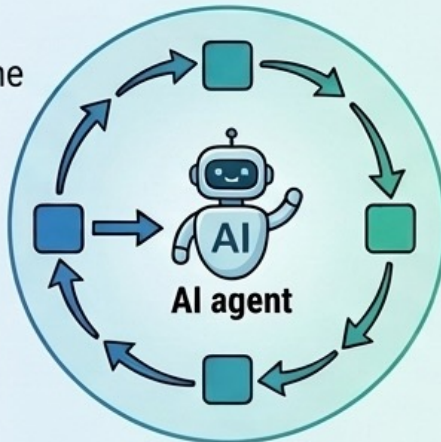


SAFETY



PREDICTABILITY

Safe zone



**LIMITED
TOOL ACCESS**



**RULES &
POLICIES**

Why Tighter Boundaries Work:

Creating safer and more predictable production systems.

OPEN-ENDED EXECUTION (FREE)



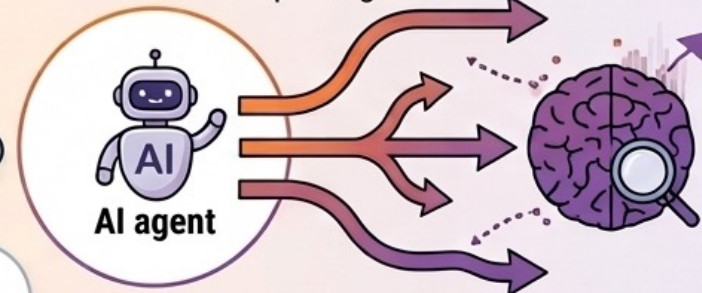
CREATIVITY



**OPEN-ENDED
PROBLEM-SOLVING**

Exploring

! INCREASED RISK



! UNPREDICTABILITY



**FULL TOOL
ACCESS**



**DELIBERATE
DECISION-MAKING**



Autonomy Level:
Where to Draw the Line?

REASONING & AUTONOMY: BOUNDED vs. UNBOUNDED AGENTS

BOUNDED REASONING



Reasons until a
**DEFINED STOPPING
CONDITION**



**HIGH SYSTEM
CONTROL**

HIGH SYSTEM CONTROL



LIMITED & PREDICTABLE COSTS



**SAFER FOR
PRODUCTION**

TIGHTER BOUNDARIES =
SAFER SYSTEMS



**CONSISTENT
OUTPUTS**

PRODUCES CONSISTENT,
PREDICTABLE RESULTS

UNBOUNDED REASONING

Reasons until
**THE MODEL
DECIDES** it is done



**MODEL-DRIVEN
AUTONOMY**



MODEL CONTROLS THE ACTIONS

**POTENTIALLY
UNBOUNDED
COSTS**



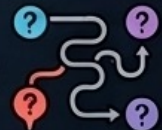
UNPREDICTABLE, POTENTIALLY
UNBOUNDED COSTS

**GREATER
EXPLORATION**

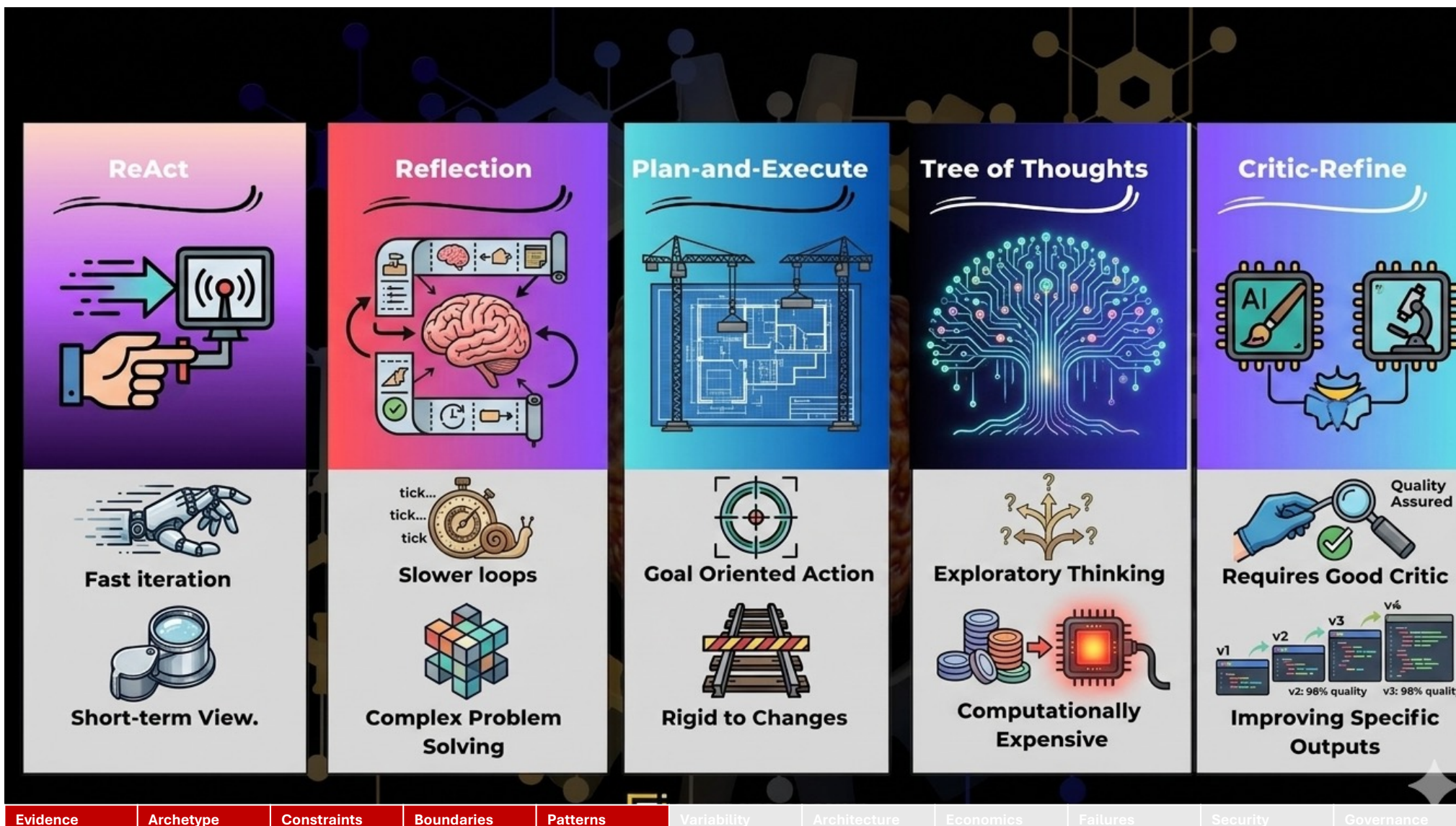


OPEN-ENDED EXECUTION
& EXPLORATION

**LESS
PREDICTABLE
BEHAVIOR**



MORE COMPLEXITY &
LESS PREDICTABILITY



DIMENSION	DETERMINISTIC AI AGENTS	STOCHASTIC AI AGENTS
Core Behavior	Same input → same output	Same input → different possible outputs
Predictability	High, consistent execution	Variable, probabilistic behavior
Control	Strong control, strictly rule-based	Flexible, sample-based, less controllable
Key Strength	Maximum reliability & precision	Creativity, reasoning & exploration
Weakness	Rigid; limited adaptability to novelty	Can be inconsistent or hallucinatory
Typical Tasks	Execution, validation, repetitive automation	Ideation, complex reasoning, content generation
Real-World Application	→ Invoice processing & data validation → DevOps deployments & infra provisioning	→ Marketing content & ad variations → Coding assistants & research summaries
Advanced Use Cases	→ Customer support order tracking → Compliance & security policy enforcement	→ Product discovery & feature ideation → Creative brainstorming from user feedback
Decision Criterion	When errors are costly and consistency is business-critical	When exploration, variety, and multiple options are valuable

Evidence	Archetype	Constraints	Boundaries	Patterns	Variability	Architecture	Economics	Failures	Security	Governance
----------	-----------	-------------	------------	----------	-------------	--------------	-----------	----------	----------	------------

5 Architectural Patterns for Effective AI Agents

A clear visual taxonomy of core architectural patterns for reliable, composable agentic systems, moving away from complex frameworks.

Linear and Specialized Patterns

Prompt Chaining

A sequence of steps where each LLM call processes the output of the previous one.

Routing

A classifier directs inputs to specialized downstream tasks or models to optimize performance.

Parallelization

Simultaneous execution of independent subtasks (**Sectioning**) or multiple attempts for higher confidence (**Voting**).

Successful AI implementations favor simple, composable patterns over complex frameworks, using augmented LLMs to orchestrate tasks through predefined paths or dynamic, model-driven decision-making.

Dynamic and Iterative Patterns

Orchestrator-workers

A central model dynamically delegates unpredictable subtasks to specialized worker models.

Evaluator-optimizer

An iterative loop where one model generates a response and another provides feedback.

Autonomous Agents

Systems where LLMs dynamically direct their own processes and tool usage in environmental loops.

Pattern Comparison by Task Complexity

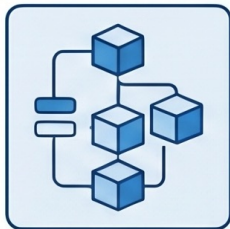
Pattern Type	Best Use Case	Primary Benefit
 Prompt Chaining	 Fixed, decomposable subtasks	 Higher accuracy via simpler calls
 Routing	 Distinct categories of input	 Optimized cost and specialization
 Orchestrator-workers	 Unpredictable, complex subtasks	 Flexibility in task delegation

Evidence	Archetype	Constraints	Boundaries	Patterns	Variability	Architecture	Economics	Failures	Security	Governance
----------	-----------	-------------	------------	----------	-------------	--------------	-----------	----------	----------	------------

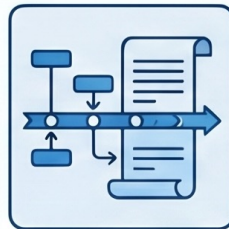
Token Economics in AI Agents

Reasoning & Long Context

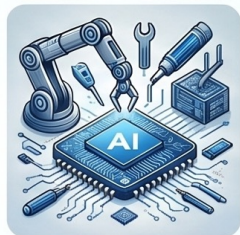
Extended reasoning steps and maintaining history/long context significantly increase the token count per request.



Reasoning steps



Long context/history



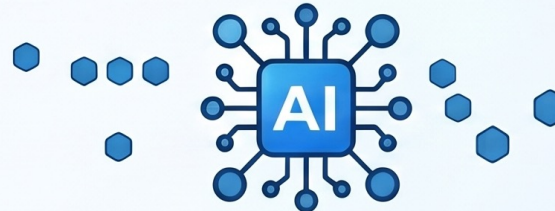
Tool Calls & Memory Retrieval

Frequent tool usage and retrieving data from memory add layers of complexity and consumption.



Inefficient Consumption (e.g., Looping, Over-tooling)

Failures like "Looping" or "Over-tooling" cause agents to repeat tool calls, exponentially increasing costs.



Cost Increases

Higher token volume directly translates to increased operational expenses for LLM API usage.



Latency & Scalability Challenges

More reasoning and tool steps lead to slower response times.



Scalability challenges

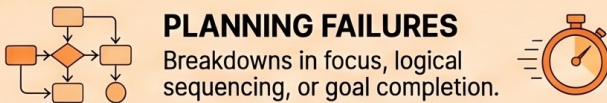
More reasoning and difficulties in scaling.

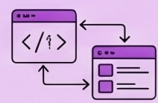
Every step an agent takes has a cost.
More reasoning and tool usage = higher cost and slower systems.

Evidence	Archetype	Constraints	Boundaries	Patterns	Variability	Architecture	Economics	Failures	Security	Governance
----------	-----------	-------------	------------	----------	-------------	--------------	-----------	----------	----------	------------

Infographic: 'A Taxonomy of AI Agent Failure Modes'

**KNOWLEDGE ERRORS**
Incorrect information or decisions based on misread or invented data.

**PLANNING FAILURES**
Breakdowns in focus, logical sequencing, or goal completion.

**CONTROL FAILURES**
Issues with tool interaction, API usage, and systemic restrictions.

Category	Failure Mode		Details	Example
Knowledge Errors	Hallucination		Agent invents facts, APIs, data, or outputs that don't exist.	LLM claims a non-existent REST endpoint returns user billing data.
	Plausible but wrong decision		Output sounds correct but reasoning/data is incorrect.	Suggests a cost optimization plan based on misread usage metrics.
Planning Failures	Goal drift		Agent gradually deviates from original objective while appearing productive.	Asked to "reduce costs," it starts deleting critical services.
	Context degradation		Agent forgets earlier constraints or instructions in long workflows.	Ignores "must use Azure region East US" after many steps.
	Premature termination		Agent stops before fully completing the task.	Generates partial report and marks as complete.
	Reasoning collapse		Logical chain starts correct but becomes inconsistent mid-way.	Starts correct debugging steps but ends with unrelated fix.
Control Failures	Looping		Repeats same actions or tool calls without progress.	Keeps calling search API for same query repeatedly.
	Tool misuse		Uses tools incorrectly or for unintended purposes.	Calls deployment API with wrong parameters causing failure.
	Over-tooling		Uses tools excessively instead of reasoning first.	Queries database repeatedly instead of caching results.
	Retry instability		Same input produces inconsistent outputs across runs.	One run approves deployment, another rejects it.
	Constraint ignorance		Ignores rules, policies, or system restrictions.	Attempts to access restricted resource despite policy.

Security Baselines for AI & Multi-Agent Systems

Don't Trust



Apply a **zero-trust mindset** to all agent **inputs** and **outputs** to prevent “**Hallucinations**” where agents invent non-existent facts or APIs.

Limit Capabilities

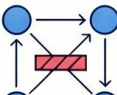


Use **granular permissions** to prevent “**Tool Misuse**” and “**Over-tooling**,” ensuring agents only access the specific resources required for their immediate task.

Verify Everything



Implement **continuous monitoring** to detect “**Goal Drift**” or “**Reasoning Collapse**” before an agent deviates from its original objective.

 Input & Prompt Security Sanitizes user prompts to prevent injection attacks that could lead to “Constraint Ignorance.”	 Output Validation Scrutinizes model responses to catch “Plausible but Wrong Doctations” based on misread metrics.	 Tool Access Control Restricts agent access to APIs to prevent “Tool Misuse” or calling deployment APIs with wrong parameters.	 Action Guardrails Hardcoded limits that stop an agent from “Goal Drift,” such as deleting critical services when asked to reduce costs.	 Orchestration Control Centralized management to prevent “Premature Termination” where agents stop before a task is fully completed.
 Identity & Access Management (IAM) Ensures every agent has a unique identity to prevent unauthorized resource access despite system policies.	 Secrets Protection Vault-based management to ensure agents do not lose or mishandle sensitive credentials during execution.	 Memory & Context Security Mitigates “Context Degradation” by ensuring instructions and constraints remain intact across long workflows.	 Observability & Logging Records all agent steps to identify “Reasoning Collapse” or inconsistent logical chains mid-way.	 Cross-Agent Validation Uses a secondary agent to check for “Knowledge Errors” and logical consistency in the primary agent’s work.
 Rate Limiting & Cost Controls Stops “Looping” failures where an agent repeatedly calls the same API without making progress.	 Model Configuration Safety Standardizes temperature and parameters to reduce “Ratry Instability” where the same input produces inconsistent outputs.	 Agent-to-Agent Trust Boundaries Establishes strict verification for data shared between agents to prevent the spread of “Hallucinated” data.	 Communication Constraints Limits the channels agents use to interact, reducing the risk of “Context Degradation” across complex teams.	 Role Separation Divides tasks between specialized agents to prevent “Over-tooling” and excessive resource querying.

Evidence	Archetype	Constraints	Boundaries	Patterns	Variability	Architecture	Economics	Failures	Security	Governance
----------	-----------	-------------	------------	----------	-------------	--------------	-----------	----------	----------	------------

Governance Baselines for AI Agents

The Three Pillars of Governance



Visibility

Establishing total transparency into the agent's internal reasoning, data inputs, and external tool interactions to prevent "black box" operations.



Control

Implementing active guardrails and system restrictions that prevent agents from ignoring policies or misusing integrated tools.



Accountability

Defining clear human ownership and oversight mechanisms to ensure every autonomous decision can be attributed to a responsible party.



Traceability

The ability to track every agent action back to the specific user prompt and original objective.



Audit Logging

Maintaining an immutable record of all reasoning steps and API calls to diagnose "reasoning collapse."



Decision Transparency

Providing a clear explanation for agent outputs to identify and correct "plausible but wrong" decisions.



HITL (Human-in-the-Loop)

Requiring human intervention for high-risk actions to prevent "goal drift" or critical service deletion.



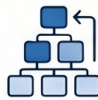
RBAC (Role-Based Access Control)

Restricting an agent's access to only the tools and data necessary for its specific assigned task.



Policy Enforcement

Programmatic constraints that prevent the agent from attempting to access restricted resources or ignoring system rules.



Versioning

Managing iterations of agent prompts, model versions, and tool configurations to ensure consistent behavior across runs.



Data Governance

Ensuring the data used for reasoning is accurate to prevent "hallucinations" or the invention of non-existent facts.



Monitoring & Alerting

Real-time detection of "looping" or "over-tooling" where agents use resources excessively without making progress.



Evaluation & Testing

Rigorous benchmarking to ensure agents do not stop prematurely or suffer from "context degradation" in long workflows.



Accountability Ownership

Identifying the specific human stakeholder responsible for the agent's actions and output quality.



Lifecycle Governance

Managing the agent's deployment from initial testing through to decommissioning to maintain performance standards.

"If you can't trace, explain, and control an agent's actions, it's not production-ready."

Evidence	Archetype	Constraints	Boundaries	Patterns	Variability	Architecture	Economics	Failures	Security	Governance
----------	-----------	-------------	------------	----------	-------------	--------------	-----------	----------	----------	------------